

ENLANG FOR DEVELOPERS

The Definitive Guide to Building Full-Stack Software with Natural English

Author: Spandan Prayas Patra

Version: 2.0.0 — Enterprise Specification Edition

Publisher: Universal EnLang Foundation Press

"Programming should be as clear, expressive, and frictionless as human thought. EnLang makes natural English a first-class, production-grade programming language across the entire software stack."

— Spandan Prayas Patra

TABLE OF CONTENTS

- Chapter 1: Introduction & Philosophical Architecture
 - Chapter 2: Variables, Optional Types & Expression Syntax
 - Chapter 3: The 3-Level Native Interactivity Architecture
 - Chapter 4: Control Flow & Iteration
 - Chapter 5: Functions, Classes, Interfaces & Async / Await
 - Chapter 6: Pattern Matching & Decision Engine
 - Chapter 7: Exception Handling & Error Control
 - Chapter 8: Built-in Collections — Lists, Arrays, Dictionaries, Sets
 - Chapter 9: String Manipulations, Math & DateTime Primitives
 - Chapter 10: File I/O, System Operations & Security
 - Chapter 11: Frontend Structural Markup (`.enlgf` → HTML5)
 - Chapter 12: Styling & Design Systems (`.enlgd` → CSS3)
 - Chapter 13: Client-Side Scripting (`.enlgs` → JavaScript ES6+)
 - Chapter 14: Database Schemas & Queries (`.enlgdb` → SQL)
 - Chapter 15: Native NLP & AI Primitives
 - Chapter 16: EnLang Web Server & Multi-File Architecture
 - Chapter 17: Complete Production Case Study — Lumina Workspace
 - Appendix A: Universal Natural Syntax Cheatsheet
 - Appendix B: CLI & EnLang Package Manager (EPM) Manual
-

CHAPTER 1: INTRODUCTION & PHILOSOPHICAL ARCHITECTURE

1.1 The Vision of Natural Programming

For over seven decades, software development has required human engineers to adapt their thoughts to artificial syntaxes, cryptic symbols (`{}`, `;`, `=>`, `::`), and unforgiving compiler rules.

EnLang was designed by **Spandan Prayas Patra** to invert this paradigm: to allow developers to write application logic in **pure, clear, expressive natural English**, while maintaining **100% deterministic compilation** into battle-tested native target languages (Python 3, HTML5, CSS3, JavaScript ES6+, SQL).

EnLang is **not** a pseudo-code generator or an LLM wrapper. It is a **deterministic, multi-target compiler and runtime engine**.

1.2 Multi-Domain Extension Architecture

Extension	Domain Purpose	Native Target	Output Format
.enlg	Core Backend Logic & Algorithms	Python 3	Executable Script
.enlgf	Frontend Structural Markup	HTML5	Semantic HTML Document
.enlgd	Design, Theme & Styling	CSS3	CSS Stylesheet Block
.enlgs	Client-Side Scripting & DOM	JS (ES6+)	JavaScript Client Script
.enlgdb	Database Schema & Queries	SQL (SQLite)	SQL DDL / DML Statements

CHAPTER 2: VARIABLES, OPTIONAL TYPES & EXPRESSIONS

2.1 Natural Expressive Flexibility

EnLang supports multiple natural syntaxes for variable assignment:

```
set score to 100
let score = 100
store 100 in score
score is 100
```

2.2 Optional Type System (v2.0 Specification)

Type annotations are completely optional:

```
define number age as 20
define decimal price as 99.50
define text username as "Spandan"
define boolean isActive as true
define list fruits as ["Apple", "Banana"]
define array tags as ["ai", "nlp"]
define dictionary profile as {"name": "Spandan"}
define set unique_ids as {101, 102}
```

CHAPTER 3: THE 3-LEVEL NATIVE INTERACTIVITY ARCHITECTURE (v2.0)

To achieve 100% zero-ambiguity execution when embedding native code expressions, EnLang defines a **Clean 3-Level Hierarchy**:

Level 1: Pure EnLang (Default)	➔ set area to width times height
Level 2: Inline Native Marker	➔ set root to @python(math.sqrt(144))
Level 3: Multi-Line Native Block	➔ python: ... end python

3.1 Level 1: Pure EnLang Syntax (Default)

Standard expressive natural English — ideal for 95% of application logic:

```
set width to 10
set height to 20
set area to width times height
display area
```

3.2 Level 2: Inline Native Markers (@python(...), @js(...), @sql(...))

When executing a single-line native expression, inline decorators provide 100% explicit, zero-ambiguity parsing across all target domains:

```
# Backend (.enlg) — Native Python Expression
import module math
set root to @python(math.sqrt(144))
display root # Output: 12.0

# Frontend Script (.enlgs) — Native JavaScript Expression
log @js(window.innerWidth)

# Database Query (.enlgdb) — Native SQL Expression
set total to @sql(SELECT count(*) FROM users)
```

3.3 Level 3: Multi-Line Native Blocks (`python: ... end python`)

For large raw code snippets (e.g. complex asyncio loops, raw SQL migrations, custom CSS rules), block directives bypass transpilation entirely:

```
python:
  import math
  x = math.factorial(5)
  print(f"Factorial of 5 is {x}")
end python
```

CHAPTER 4: CONTROL FLOW & ITERATION

```
if score is greater than 90 then:
  display "Grade: A+"
otherwise if score is greater than 80 then:
  display "Grade: A"
else:
  display "Grade: B"

repeat 5 times do:
  display "Processing batch item..."

for each user in users do:
  display "User: " plus user
```

CHAPTER 5: FUNCTIONS, CLASSES, INTERFACES & ASYNC / AWAIT

5.1 Enterprise Interfaces & Implements (v2.0)

```
interface Authenticatable:
  function login(credentials)
  function logout()
end interface

class UserSession implements Authenticatable:
  function login(credentials):
    display "User authenticated"

  function logout():
    display "Session closed"
end class

s = UserSession()
display str(isinstance(s, Authenticatable)) # True
```

5.2 Async & Await Primitives (v2.0)

```
import module asyncio

async function fetch_user_data(user_id):
    sleep 0.1 seconds
    return "Profile Data for ID " plus str(user_id)

async function main():
    set profile to await fetch_user_data(101)
    display profile

asyncio.run(main())
```

CHAPTER 6: PATTERN MATCHING (match / case / default)

```
set role to "admin"

match role:
    case "admin":
        display "Full System Access"
    case "editor", "author":
        display "Content Modification Access"
    case is greater than 50:
        display "Score Requirement Met"
    default:
        display "Guest Access Only"
end match
```

CHAPTER 7: EXCEPTION HANDLING (try / except / finally)

```
try:
    set result to 100 divided by 0
except:
    display "Caught runtime exception"
finally:
    display "Cleanup block executed"

raise ValueError with message "Input out of bounds"
throw error "Authentication failure"
```

CHAPTER 17: PRODUCTION CASE STUDY — LUMINA WORKSPACE

server.enlg

```
# Using Level 2 Inline Native Marker & Level 3 Block
set version_tag to @python("v" + str(2.0))

python:
  print(f"Booting Lumina Server {version_tag}...")
end python

start web server on port 8000
```

APPENDIX A: UNIVERSAL SYNTAX REFERENCE

Feature Level	EnLang Syntax Example	Native Transpilation
Level 1 (Pure)	set area to width times height	area = width * height
Level 2 (Inline Marker)	set val to @python(math.sqrt(25))	val = math.sqrt(25)
Level 3 (Native Block)	python: ... end python	Verbatim Block Execution
Interface	interface Authenticatable: ... end interface	Abstract Contract Class
Implements	class UserSession implements Authenticatable:	Multi-Inheritance Class
Async Func	async function fetch_data():	async def fetch_data():
Await Expr	set res to await fetch_data()	res = await fetch_data()
Pattern Match	match role: case "admin": default: end match	Non-fallthrough Branching

APPENDIX B: CLI & PACKAGE MANAGER (EPM) MANUAL

```
enlang run app.enlg           # Compiles & executes backend logic
enlang server --port 8000     # Launches zero-config HTTP host server
enlang add py:requests        # Installs Python PyPI package
enlang add web:chart.js       # Installs Web NPM package
```

Copyright © 2026 Spandan Prayas Patra. All rights reserved.

Published under the Open EnLang Specification License.